UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación, Sede Sololá.
Ingeniero Wilmer Orlando Xeron
Base de Datos 1 I Sec. “A”

# Fase 2: Ingesta Inteligente

Cristhian Cesar Felipe Mucún Queché
2290-17-17851

Panajachel 30 de septiembre de 2026

```
-- ERP CORE SYSTEM - FASE 2: DML MASIVO, CONJUNTOS Y UDFs
-- Alumno: Cristhian Mucun 2290-17-17851
USE ERP_Corporativo;

/*
 *  Primero comprobemos si existen los datos
*/
SELECT 'Sucursal' AS Entidad, COUNT(*) AS Existe
FROM glb_Sucursales
WHERE IdSucursal = 1

UNION ALL

SELECT 'Departamento Empresa', COUNT(*)
FROM hr_DepartamentosEmpresa
WHERE IdDeptoEmpresa = 1

UNION ALL

SELECT 'Puesto', COUNT(*)
FROM hr_Puestos
WHERE IdPuesto = 1;

/*
```

```
 * Como el laboratorio posteriomente utiliza IdSucursal = 1, etc,
vamos a preparar un conjunto mínimo choerente.
 */


-- Moneda
INSERT INTO glb_Monedas (
    CodigoISO,
    NombreMoneda,
    TasaCambioBase
)
VALUES (
    'GTQ',
    'Quetzal',
    1.0000
);

-- Pais
INSERT INTO glb_Paises (NombrePais)
VALUES ('Guatemala');

--Departamento
INSERT INTO glb_Departamentos(
        IdPais,
        NombreDepto
)
VALUES (
```

```
        1,
        'Guatemala'
);

-- Municipio
INSERT INTO glb_Municipios (
        IdDepto,
        NombreMunicipio
)
VALUES (
        1,
        'Guatemala'
);

-- Empresa
INSERT INTO glb_Empresas (
    NIT,
    RazonSocial,
    IdMonedaLocal
)
VALUES (
    '1234567-8',
    'ERP Corporativo Guatemala',
    1
);
```

```
-- Sucursal
INSERT INTO glb_Sucursales (
    IdEmpresa,
    IdMunicipio,
    NombreSucursal,
    EsMatriz
)
VALUES (
    1,
    1,
    'Sucursal Central',
    1
);

-- Departamento Empresarial
INSERT INTO hr_DepartamentosEmpresa (
    NombreDepartamento
)
VALUES (
    'Administración'
);

-- Puesto
INSERT INTO hr_Puestos (
```

```
    NombrePuesto,
    IdPuestoJefe,
    SalarioBase
)
VALUES (
    'Cajero',
    NULL,
    4500.00
);

-- Categoria de inventario
INSERT INTO inv_Categorias (NombreCategoria)
VALUES
    ('Ferretería'),
    ('Construcción'),
    ('Electricidad');

-- Insertamos Empleados
SET IDENTITY_INSERT hr_Empleados ON; --Proporcionaremos manualmente un valor para la columna IDENTITY

INSERT INTO hr_Empleados (
    IdEmpleado,
    IdSucursal,
    IdDeptoEmpresa,
    IdPuesto,
```

```
    DPI,
    NombreCompleto,
    FechaNacimiento,
    FechaIngreso
)
VALUES
( 1, 1, 1, 1, '1234567890101', 'Carlos Roberto Soto', '1985-05-15', '2015-02-01' ),
( 2, 1, 1, 1, '9876543210101', 'María Fernanda López', '1992-11-20', '2020-08-15' ),
( 3, 1, 1, 1, '5555555550101', 'Luis Armando Pérez', '1998-01-10', '2024-01-05' );

SET IDENTITY_INSERT hr_Empleados OFF; --Vuelve al comportamiento normal de IDENTITY

SELECT
    e.IdEmpleado,
    e.NombreCompleto,
    e.DPI,
    s.NombreSucursal,
    d.NombreDepartamento,
    p.NombrePuesto,
    e.Estado
FROM hr_Empleados e
INNER JOIN glb_Sucursales s
    ON e.IdSucursal = s.IdSucursal
```

```
INNER JOIN hr_DepartamentosEmpresa d
   ON e.IdDeptoEmpresa = d.IdDeptoEmpresa
INNER JOIN hr_Puestos p
   ON e.IdPuesto = p.IdPuesto
ORDER BY e.IdEmpleado;

--El poder del MERGE
CREATE TABLE #Staging_ProductosCSV (
   SKU VARCHAR(50) COLLATE database_default,
   IdCategoria INT,
   Descripcion VARCHAR(200) COLLATE database_default,
   PrecioVentaBase DECIMAL(12,2)
);

INSERT INTO #Staging_ProductosCSV
VALUES
('FERR-001', 1, 'Martillo de Acero Forjado Pro', 125.50),
('ELEC-002', 3, 'Cable Calibre 12 THHN 100m', 450.00),
('CONS-005', 2, 'Bolsa de Cemento Gris 50kg', 75.00);

INSERT INTO inv_Productos (
   IdCategoria,
   SKU,
   Descripcion,
   PrecioVentaBase
```

```
)
VALUES
(1, 'FERR-001', 'Martillo Viejo', 95.00);

INSERT INTO inv_Productos (
    IdCategoria,
    SKU,
    Descripcion,
    PrecioVentaBase
)
VALUES
(1, 'FERR-999', 'Producto Descontinuado', 10.00);

MERGE inv_Productos AS TARGET

USING #Staging_ProductosCSV AS SOURCE

ON TARGET.SKU = SOURCE.SKU

WHEN MATCHED THEN
    UPDATE SET
        TARGET.IdCategoria = SOURCE.IdCategoria,
        TARGET.Descripcion = SOURCE.Descripcion,
        TARGET.PrecioVentaBase = SOURCE.PrecioVentaBase,
        TARGET.Activo = 1
```

```
WHEN NOT MATCHED BY TARGET THEN
    INSERT (
        IdCategoria,
        SKU,
        Descripcion,
        PrecioVentaBase
    )
    VALUES (
        SOURCE.IdCategoria,
        SOURCE.SKU,
        SOURCE.Descripcion,
        SOURCE.PrecioVentaBase
    )

WHEN NOT MATCHED BY SOURCE THEN
    UPDATE SET
        TARGET.Activo = 0;

DROP TABLE #Staging_ProductosCSV;

SELECT
    SKU,
    Descripcion,
    PrecioVentaBase,
```

```
    Activo
FROM inv_Productos
ORDER BY SKU;

-- Minería rapida con Conjuntos Excluyentes
INSERT INTO pos_TiposCliente (NombreTipo)
VALUES ('Consumidor Final');

INSERT INTO pos_Clientes (
    IdTipoCliente,
    NombreCompleto
)
VALUES (
    1,
    'C/F'
);

INSERT INTO pos_VentasMaster (
    IdVenta,
    IdDepto,
    IdSucursal,
    IdCliente,
    IdEmpleadoCajero,
    TotalNeto
)
```

```
VALUES
    (1, 1, 1, 1, 1, 500.00),
    (2, 1, 1, 1, 2, 1500.00);

SELECT
    IdEmpleado,
    NombreCompleto
FROM hr_Empleados
WHERE Estado = 'Activo'

EXCEPT

SELECT
    E.IdEmpleado,
    E.NombreCompleto
FROM hr_Empleados E
INNER JOIN pos_VentasMaster V
    ON E.IdEmpleado = V.IdEmpleadoCajero;

-- Funciones definidas por el usuario (UDF)
CREATE OR ALTER FUNCTION hr.ufn_CalcularEdadExacta (
    @FechaNacimiento DATE
)
RETURNS INT
AS
```

```
BEGIN
   DECLARE @Edad INT;

   SET @Edad =
      DATEDIFF(YEAR, @FechaNacimiento, GETDATE())
      -
      CASE
         WHEN (MONTH(@FechaNacimiento) >
MONTH(GETDATE()))
            OR (
               MONTH(@FechaNacimiento) = MONTH(GETDATE())
               AND DAY(@FechaNacimiento) > DAY(GETDATE())
            )
         THEN 1
         ELSE 0
      END;

   RETURN @Edad;
END;

SELECT
   DPI,
   NombreCompleto,
   FechaNacimiento,
   hr.ufn_CalcularEdadExacta(FechaNacimiento) AS EdadReal
FROM hr_Empleados;
```

```
CREATE OR ALTER FUNCTION
inv.ufn_EstadisticasInventarioPorCategoria
(
    @CategoriaID INT
)
RETURNS TABLE
AS

RETURN (
    SELECT
        c.NombreCategoria,
        COUNT(p.IdProducto) AS TotalSKUs,
        MIN(p.PrecioVentaBase) AS PrecioMasBajo,
        MAX(p.PrecioVentaBase) AS PrecioMasAlto,
        AVG(p.PrecioVentaBase) AS PrecioPromedioMercado

    FROM inv_Categorias c

    LEFT JOIN inv_Productos p
        ON c.IdCategoria = p.IdCategoria
        AND p.Activo = 1

    WHERE c.IdCategoria = @CategoriaID

    GROUP BY c.NombreCategoria
```

);

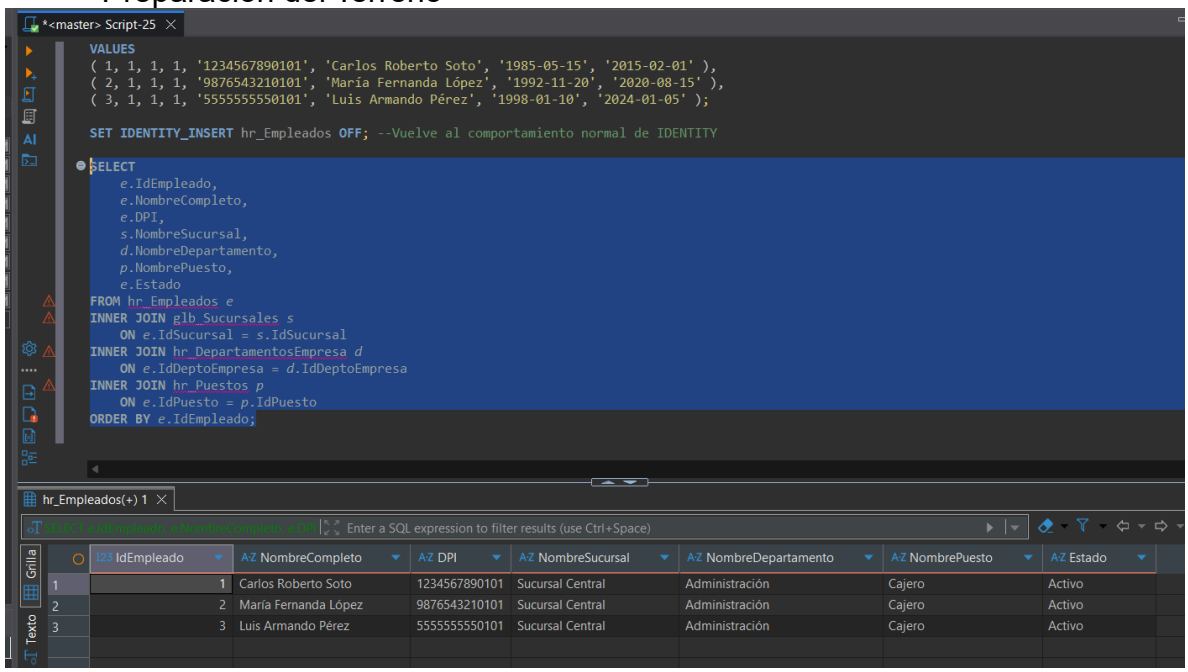
**SELECT ***

**FROM inv.ufn_EstadisticasInventarioPorCategoria(1);**

## Análisis crítico empresarial:

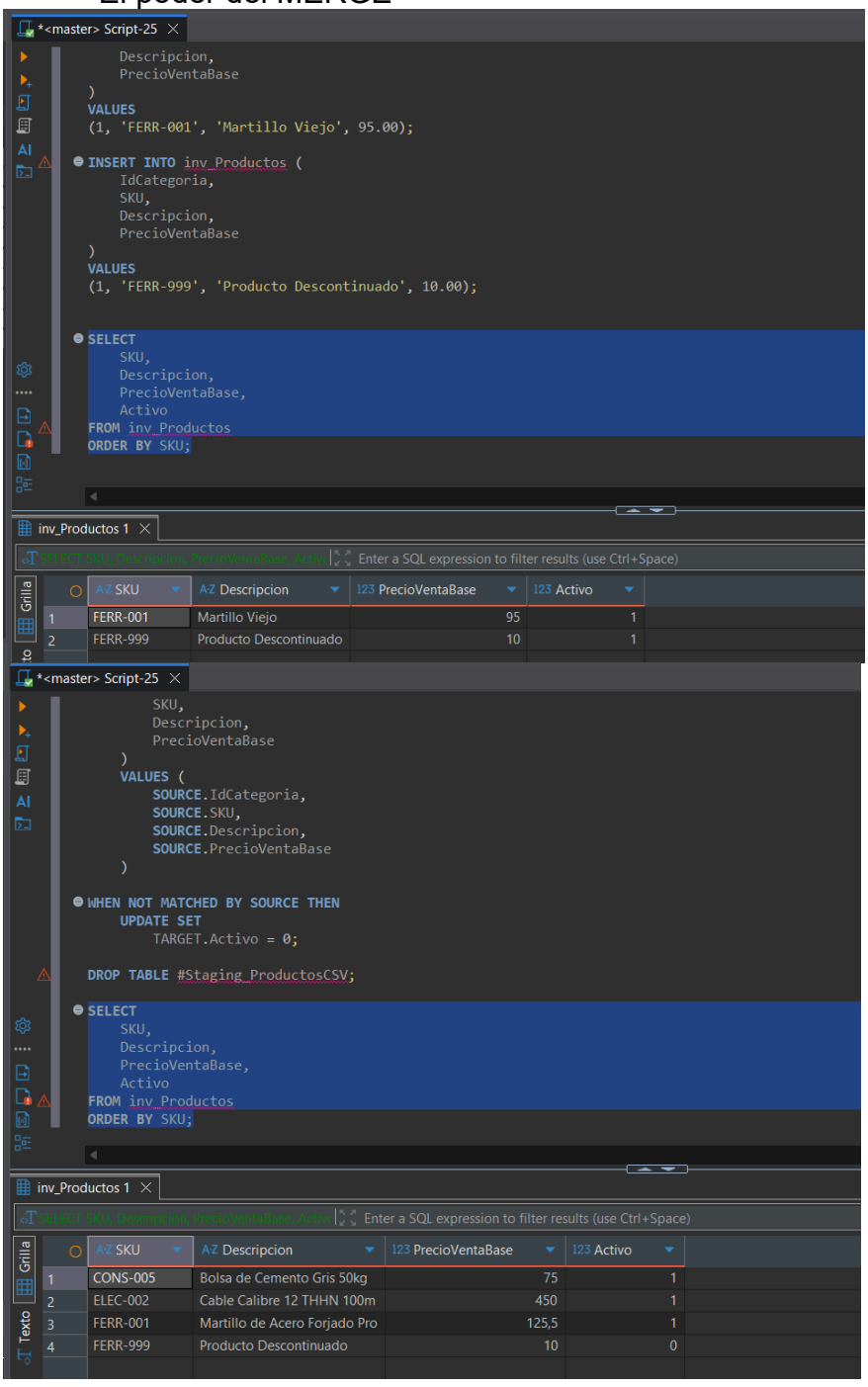
### Ingesta Inteligente:

- Preparación del Terreno




Preparamos los datos maestros necesarios para ejecutar correctamente las operaciones de la Fase 2. El script original inserta categorías y tres empleados con identificadores específicos mediante IDENTITY_ INSERT, pero asume previamente la existencia de la Sucursal 1, Departamento empresarial 1 y Puesto 1, para evitar violaciones de claves foráneas, se completo primero la cadena de dependencias creando una moneda, país, departamento geográfico, municipio, empresa, sucursal, departamento interno y puesto válido, posteriormente se insertaron las categorías requeridas y se habilitó temporalmente IDENTITY_INSERT para conservar los identificadores 1, 2 y 3 de los empleados utilizados por los ejercicios posteriores. Finalmente se deshabilitó esta opción para devolver la tabla a su comportamiento automático normal. En lugar de ignorar las claves foráneas como simplificación del laboratorio, se crearon explícitamente los registros padres requeridos. De esta forma, las pruebas posteriores trabajan sobre un modelo relacional válido y no dependen de datos inexistentes.

- El poder del MERGE

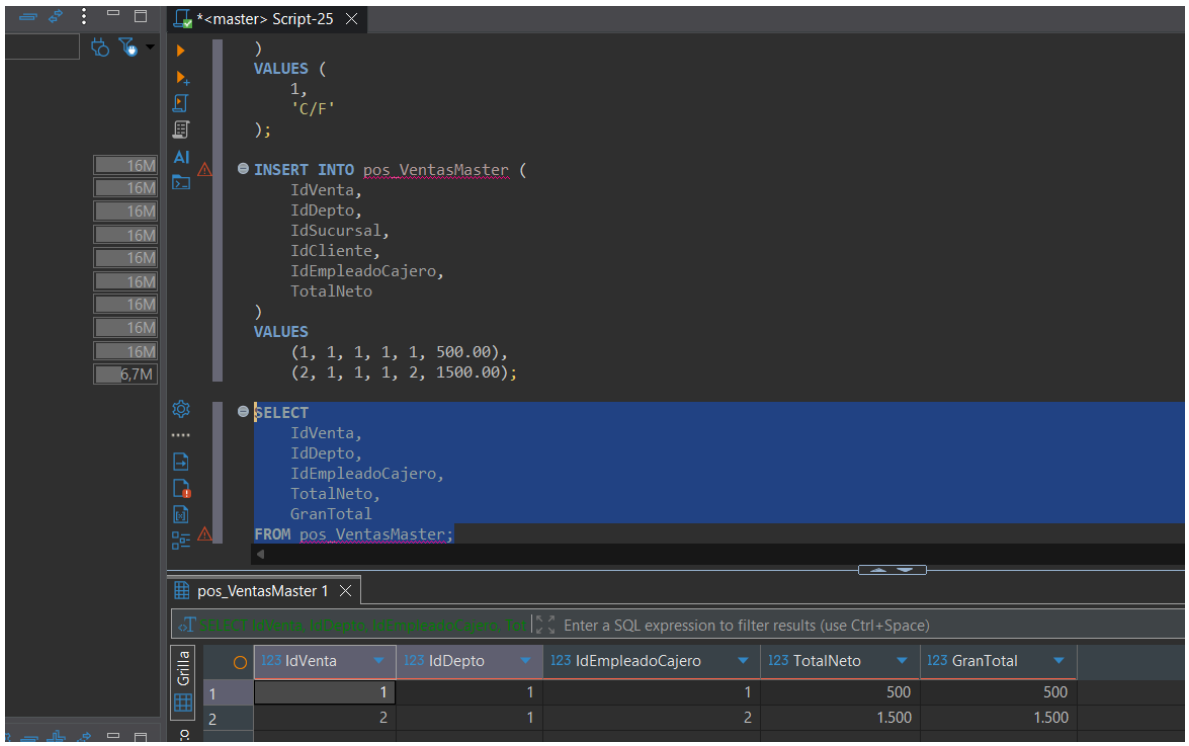


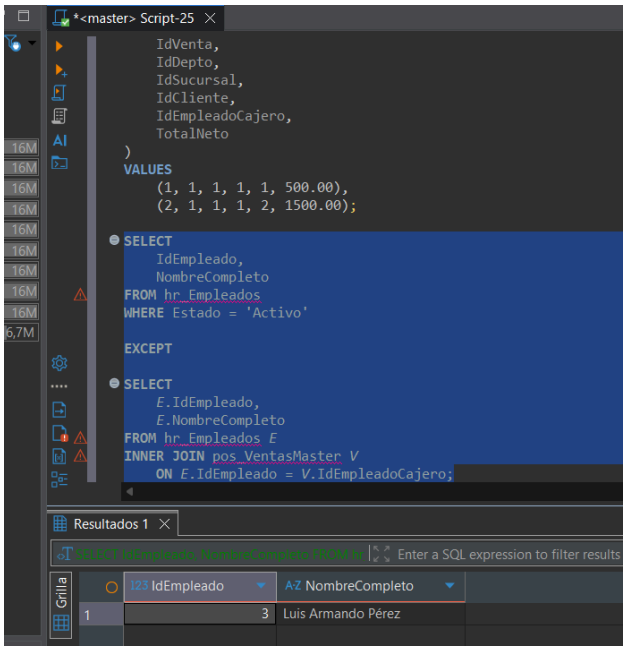

Simulamos la sincronización de un catálogo de productos recibido desde un archivo externo, la información se carga primero en una tabla temporal de staging, evitando modificar directamente la tabla productiva mientras se prepara la comparación, mediante MERGE el SKU funciona como llave de coicidencia entre la fuente e inv_Productos, los productos existentes se actualizan, los nuevos se isertan y los que dejan de aparecer se desactivan mediante baja lógica en lugar de eliminarse

físicamente, conservando así su historial, durante el análisis se mejoró la condición WHEN MARCHED para actualizar también la categoría y reactivar el producto cuando vuelve a aparecer en el catálogo, ya que el script original actualizaba únicamente descripción y precio y podía dejar un producto vigente con Activo = 0. La regla WHEN NOT MATCHED BY SOURCE es segura únicamente si el archivo recibido representa el catálogo completo vigente, si el archivo contuviera solamente los productos modificados durante la semana, esta lógica desactivaría incorrectamente todos los demás productos por ello, antes de aplicar este patrón en producción debe definirse si la fuente es un catálogo completo o una carga incremental.

- Minería Rápida con Conjuntos Excluyentes

Para este tipo de comparación de conjuntos, EXCEPT resulta más seguro y expresivo frente al comportamiento de NOT IN cuando la subconsulta puede contener valores NULL.

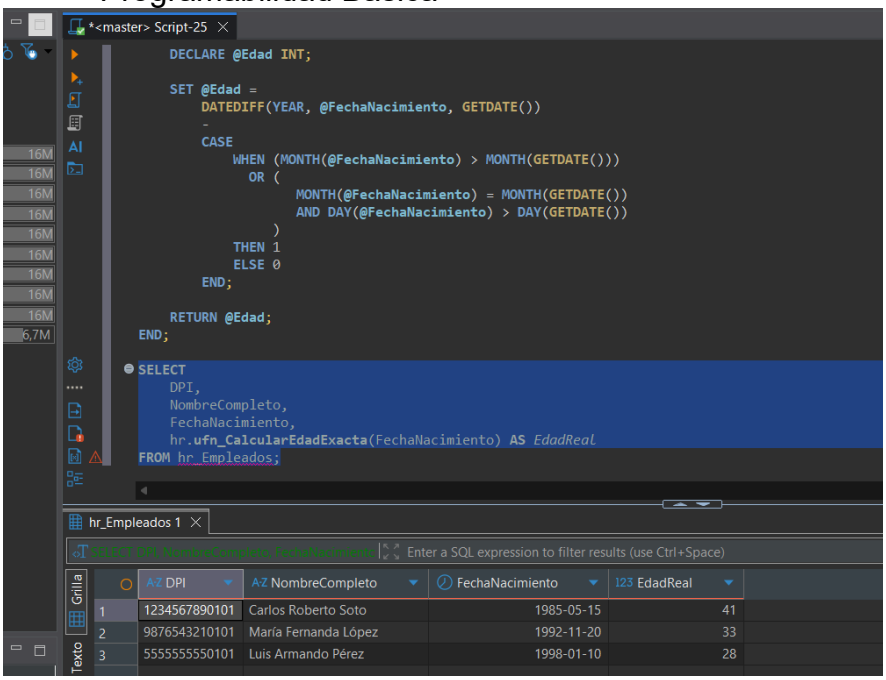


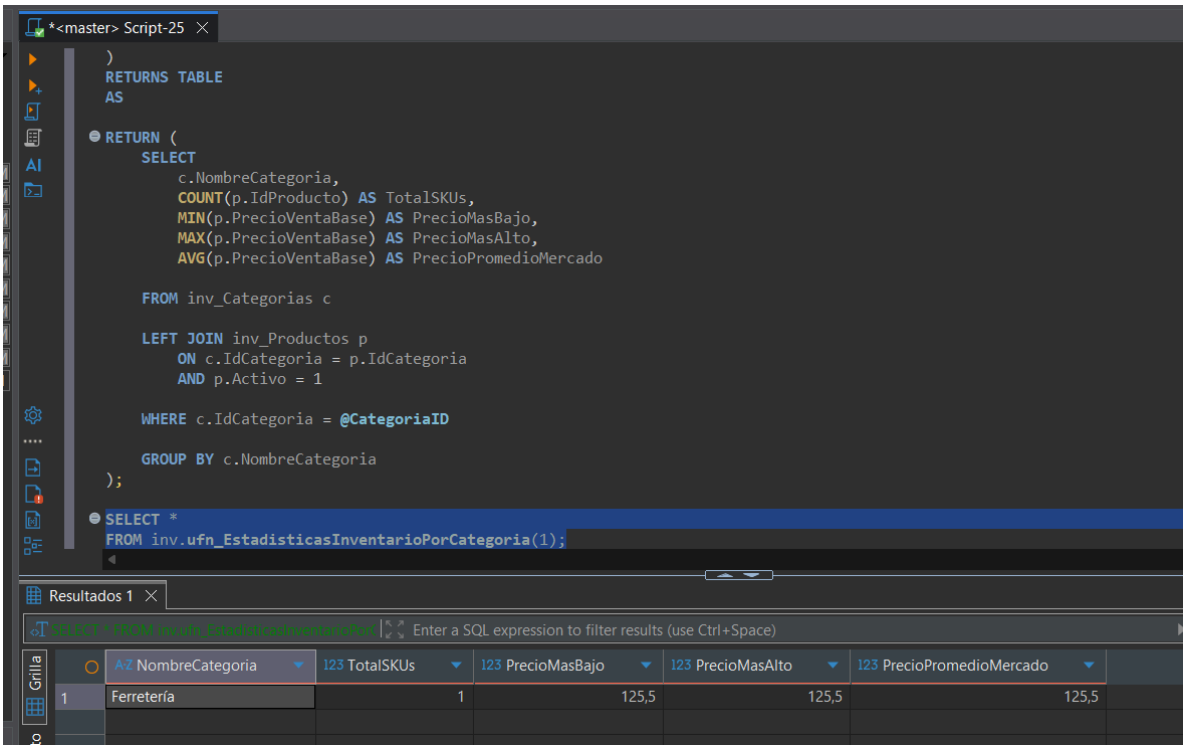



Utilizamos algebra de conjuntos para identificar empleados activos que todavía no han registrado ventas. Primero se preparó un cliente de prueba y se registraron dos ventas asociadas únicamente a los empleados 1 y 2, posteriormente EXCEPT compara el conjunto completo de empleados activos contra el conjunto de empleados presentes en pos_VentasMaster, devolviendo únicamente los registros que existen en el primer conjunto y no en el segundo. El resultado identifica correctamente al empleado 3 como personal sin ventas registradas, la consulta aprovecha las relaciones creadas en la Fase 1 y no requirió cambios adicionales, ya que las ventas cumplen los controles de integridad agregados previamente.

Si usamos EXCEPT en ligar de depender de NOT IN porque una subconsulta que contenga valores NULL puede provocar resultados indeterminados con NOT IN.

- Programabilidad Básica






Se creó la función escalar hr.ufn_CalcularEdadExacta para obtener la edad de un empleado a partir de su fecha de nacimiento sin almacenar un dato que cambia con el tiempo. La función utiliza DATEDIFF para calcular la diferencia inicial de años y posteriormente verifica mediante un CASE si el cumpleaños del año actual todavía no ha ocurrido, restando una unidad cuando corresponde. De esta manera, la lógica se centraliza en SQL Server y puede reutilizarse en consultas y reportes sin repetir el cálculo en cada aplicación. No fue necesario modificar el código original para el escenario planteado, aunque casos especiales como nacimientos el 29 de febrero requerirían una regla empresarial explícita si fueran relevantes para el sistema.

Se creó una función tabular en línea que recibe el identificador de una categoría y devuelve estadísticas de sus productos activos mediante COUNT, MIN, MAX y AVG. Durante la revisión se detectó que el código original utilizaba un LEFT JOIN pero colocaba p.Activo = 1 en la cláusula WHERE, lo que eliminaba las categorías que no tuvieran productos activos y anulaba parcialmente el propósito del LEFT JOIN. Se corrigió trasladando esta condición al ON, permitiendo que una categoría válida continúe apareciendo con TotalSKUs = 0 aunque no tenga productos activos. Con este ajuste, la función diferencia correctamente entre una categoría existente sin inventario activo y una categoría inexistente.